

When you invoke `makeIncrementer`, you get back a function that increments the value of the variable `n` and returns the value:

```
incrementer = makeIncrementer()
incrementer()
# 1

incrementer()
# 2
```

Is it an object?

If closures are new to you but you're comfortable with objects, then you might think of a closure as an object. Suppose you had an `incrementer` object with a single method called `increment` that incremented an internal value and returned it. This would be essentially the same thing as the closure demonstrated in this section.

The returned function has access to the variables scoped to `makeIncrementer`. The returned function *closes over* the variables in scope, resulting in *closure*. Each time `makeIncrementer` is invoked, a new closure is created, meaning you can have several of them:

```
up = makeIncrementer()
oneMore = makeIncrementer()
up()
# 1

up()
# 2

up()
# 3
```